

Oxivolve: Performance-aware, AI-assisted C-to-Rust Transpilation

1st Kunal Pai

Department of Computer Science
UC Davis
kunpai@ucdavis.edu

2nd Dalton Davis

Department of Computer Science
UC Davis
drdavis@ucdavis.edu

3rd Yubo Bai

Department of Computer Science
UC Davis
gabai@ucdavis.edu

4th Alex Gao

Department of Computer Science
UC Davis
alwgao@ucdavis.edu

5th Tapti Palit

Department of Computer Science
UC Davis
tpalit@ucdavis.edu

Abstract—AI-assisted C-to-Rust transpilation pipelines focus primarily on correctness and safety, routinely overlooking runtime performance. We address this gap by first showing that simply prompting agents for performant Rust code is insufficient: no single translation strategy is universally optimal, and both language-specific and algorithm-level decisions can significantly affect performance. We propose a multi-variant generation framework in which an AI agent produces multiple distinct Rust implementations for each C program and selects the best performing variant. We evaluate two strategies: multi-variant generation from C source, and summary-guided generation that first abstracts the C code to encourage more diverse implementations. Finally, we introduce Oxivolve, an agentic optimization approach that refines generated Rust code by using runtime performance feedback. Evaluated on four microbenchmarks and six macrobenchmarks across three state-of-the-art coding agents, Oxivolve achieves the fastest runtime among all strategies on 6 of 10 benchmarks with Claude Opus 4.6 while maintaining perfect correctness (geometric mean speedup of $2.20\times$ over C), and on 4 of 10 with Gemini 3 Pro ($1.64\times$, 90% correctness).

I. INTRODUCTION

Rust is a memory-safe programming language that has gained significant popularity in recent years. This popularity stems in part from the prevalence of memory-safety bugs, which include spatial violations (e.g., out-of-bounds accesses) and temporal violations (e.g., use-after-free and double frees), and which account for a large share of security vulnerabilities. Rust’s type system and ownership model rule out these classes of errors at compile time, driving its adoption in popular software projects such as Firefox [59] and Linux [75], and prompting many organizations to actively migrate their C codebases to Rust [54]. This has spurred significant interest in automated C-to-Rust transpilation, with recent works proposing a variety of approaches, ranging from syntactic translation [1], [25], [24] to LLM-based ones [80], [26], [8], [71], [63], [92]. Concurrently, general-purpose coding agents have also begun gaining popularity for code generation tasks [9], [44]. As LLMs and autonomous coding agents grow more accurate, they offer a powerful mechanism for scaling codebase migrations to Rust.

AI-based C-to-Rust transpilation toolchains [6], [92], [63], [8], [18] typically focus primarily on functional correctness. However, correctness is not the only concern when migrating C codebases to Rust. Performance is especially important for Rust because the promise of C-like performance is what enables its use for systems software. However, some of Rust’s safety checks are performed at runtime. This includes bounds checks for *slices* and *vectors*, as well as checks related to smart pointers, such as *Rc* and *RefCell*. Therefore, performance parity between the original C code and the transpiled Rust code is not guaranteed.

Migrating codebases often gives developers the opportunity to make performance optimizations that a literal, line-by-line translation would miss. In the case of C-to-Rust transpilation, this opens up at least two avenues for improving performance. First, the developer can correct latent inefficiencies in the legacy C source, which, as Nistor [62] demonstrates, frequently contains hidden performance bugs that escape traditional testing. Second, the developer can exploit idiomatic Rust paradigms such as iterators and generics that allow the compiler to elide the language’s runtime safety checks. For example, a developer might replace an $O(N^2)$ insertion sort with an $O(N \log N)$ merge sort, or restructure C-style index-based loops into iterators, which often compile down to bounds-check-free code. In this work, we investigate AI-based techniques to leverage migration in the same way to rewrite and rearchitect code for performance and efficiency.

To succeed, AI must treat the C implementation as *functional* guidance and avoid its syntax-level details in order to actively restructure the logic into idiomatic Rust. This paradigm aligns with established High-Level Synthesis (HLS) transpilation methodologies [36], [84], [87] where a high-level specification is used to drive automated, optimized code generation. However, while recent automated C-to-Rust transpilation tools [6], [92], [63], [8], [39], [38], [40] perform limited, *local* rewrites to improve idiomaticity, no current C-to-Rust pipeline focuses on rewriting or redesigning large individual components, to the best of our knowledge. Indeed,

as we discuss in §V, models frequently mirror the C source syntax closely, causing them to struggle to break free from systemic inefficiencies embedded within the legacy code.

To overcome these limitations, we investigate different *steering mechanisms* to guide AI-agents to produce more efficient Rust transpilations from legacy C code. These steering mechanisms lie on a spectrum, ranging from simple prompt-based approaches that still ground the AI-agent in the C implementation to high-level summary-based approaches where the AI-agent has significant freedom in choosing the system design. First, we investigate a simple steering mechanism: instructing AI to prioritize performance when transpiling the C codebase (e.g., embedding “make the code performant” in the prompt). Our experiments show that this simple prompt-based approach *severely underperforms* the original C code, suggesting that AI agents are not capable of generating performant Rust code when explicitly prompted to do so.

Furthermore, we observe that no single transpilation strategy is universally optimal. For instance, while a Rust loop can be implemented using either iterators or index-based access, the ‘best’ choice is application-specific, i.e., iterators do not consistently outperform indexing, or vice versa. To achieve high performance, transpilation must incorporate application-specific reasoning that accounts for these trade-offs. Therefore, appending generic directives to a prompt fails to guide AI-agents toward truly efficient code.

Since no single strategy is universally optimal, it is imperative to explore multiple strategies and select the best-performing one. Inspired by multi-variant execution, a long-standing approach for improving reliability and fault tolerance [5], [14], [41] recently adapted for LLM code generation [68], we explore a multi-variant generation steering mechanism for transpilation and performance optimization, in which the AI agent produces multiple distinct Rust implementations for each C function and selects the best-performing one. Specifically, we propose two variant-generation strategies: (1) direct generation from the C source code, and (2) summary-driven generation, which first abstracts the C code into a high-level summary. The intuition behind the latter is that summarization forces the AI to treat the source code merely as algorithmic guidance and inspires it to explore more diverse, optimized rewrites. Our evaluation shows that direct multi-variant generation from the C source code leads to significant performance improvements, but providing the AI agent with just the summary results in many functional correctness errors (§V).

Finally, we propose *Oxivolve*, a steering mechanism that aims to strike a balance between the direct C-based and the abstract summary-based steering mechanisms. *Oxivolve* is based on agentic evolution applied over the multi-variant approach [68], [5], [14], [82]. The system leverages an agentic feedback loop, executing candidate variants and forwarding their precise performance metrics back to the agent as semantic context to synthesize a single, optimized recombination. By treating this descriptive performance feedback as an optimization signal, the agent acts as an implicit Pareto selector [28]

that identifies and combines the best features without requiring hard-coded crossover and mutation rules, as with traditional evolutionary algorithms [17]. Our evaluation shows that *Oxivolve* achieves competitive runtime performance among all strategies (§V).

In summary, this paper makes the following contributions:

- We show that simply prompting for performant code does not yield measurable improvement, and that no single optimization strategy is universally optimal.
- We show that multi-variant generation significantly improves the performance of AI-transpiled Rust code.
- We propose *Oxivolve*, an evolutionary steering mechanism that uses runtime and memory usage as feedback and agents as an implicit Pareto selector to recombine candidate features into optimized Rust code.
- We perform a post-hoc empirical validation of the microarchitectural claims made by the coding agents. We find that while they can produce optimized code, they frequently provide misguided justifications for their performance gains, highlighting the need for careful, automated performance testing as part of the transpilation toolchain.

II. MOTIVATING EXAMPLE: MATRIX MULTIPLICATION

To illustrate the performance diversity space and emphasize the necessity of dedicated steering mechanisms for performance, we present three performance-diverse versions of the simple but widely-used matrix multiplication use case [60], [61], [91], [32].

Approach 1: The Naive $i - j - k$ Baseline. Given two matrices, computing the product intuitively translates to a triply-nested loop: two outer loops to iterate over the entries of the result matrix, and a third inner loop to calculate the dot product of the corresponding row and column (shown in Listing 1). A brief survey of open-source repositories reveals that this exact naive approach in C is frequently published for educational purposes [4] and serves as the worst-case baseline in systems benchmarking suites [11].

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        C[i][j] = 0;
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Listing 1. Naive $O(N^3)$ Matrix Multiplication

Since LLMs rely heavily on these open-source repositories for their training corpora, this mathematically intuitive implementation is deeply ingrained in their learned representations. Consequently, when state-of-the-art models are tasked with generating code for matrix operations, they typically default to this naive $O(N^3)$ implementation. Furthermore, if a developer provides this naive code as a prompt to an LLM for translation or refactoring, the model typically preserves the suboptimal

nested-loop structure rather than introducing a performance optimization due to anchoring bias [77].

This anchoring bias is particularly detrimental when translating such algorithms from C to Rust. In C, the naive indexing in Listing 1 translates directly to raw memory offset calculations. However, in Rust, direct slice indexing includes a runtime bounds check to guarantee spatial memory safety and prevent buffer overruns. When an LLM naively translates the triply-nested loop structure from C to Rust, it inadvertently adds this bounds-checking overhead. Similar sources of overhead are likely to appear in other workloads that undergo a straightforward or naive translation.

Approach 2: Iterator-Based Traversal. Breaking away from the C-style indexing pattern established in Listing 1 and re-imagining the implementation in terms of Rust’s idioms and capabilities can enable true zero-cost abstraction. Specifically, using iterators instead of manual indexing (Listing 2) eliminates bounds-checking overhead entirely. Precomputing the columns of B into `cols_b` aims to ensure contiguous memory access during the inner dot product [56]. However, constructing `cols_b` requires N distinct dynamic allocations, incurring significant runtime overhead alongside an $O(N^2)$ space complexity for $N \times N$ matrices.

Listing 2. Iterators and Precomputed Columns

```
fn multiply_matrices(
    a: &[Vec<i32>], b: &[Vec<i32>],
    c: &mut [Vec<i32>], size: usize
) {
    let cols_b: Vec<Vec<i32>> = (0..size)
        .map(|j| {
            b.iter().map(|r| r[j]).collect()
        }).collect();
    c.iter_mut().enumerate()
        .for_each(|(i, row)| {
            row.iter_mut().enumerate()
                .for_each(|(j, v)| {
                    *v = a[i].iter()
                        .zip(&cols_b[j])
                        .map(|(x, y)| x * y)
                        .sum();
                });
        });
}
```

Approach 3: Cache-Aware $i - k - j$ Loop Reordering. An alternative strategy involves retaining the manual loop structure from **Approach 1** but reordering the traversal to an $i - k - j$ loop order. Unlike the iterator method, this approach requires no additional memory overhead. Instead, it exploits microarchitectural characteristics by maximizing sequential memory operations, thus improving cache locality and reducing runtime.

Ultimately, these three variations highlight trade-offs in runtime, memory consumption, and microarchitectural efficiency. They demonstrate why performance cannot be treated as an afterthought in AI-assisted code generation. Since models are statistically predisposed to replicate the computationally inefficient paradigms that dominate their training corpora [77],

relying on zero-shot translation for systems programming yields degraded performance. Furthermore, because the optimal algorithm is application-specific, a simple, standardized prompt approach is not sufficient to correct these biases. Instead, we must develop systematic steering strategies and evaluation frameworks to guide AI to bypass the syntax of the source language and systematically explore a diverse search space to discover the most performant, idiomatic translation.

III. SYSTEM DESIGN

A. Steering Mechanisms

In this section, we describe the steering mechanisms we use to guide coding agents toward generating performant Rust code from C source. Starting with simple prompt focused approaches, we progressively grant the AI agent more freedom to explore different implementations, guiding the model toward more performant output.

1) *Base Prompt:* We leverage the known-good prompt of the RustAssure [6] toolchain, focusing on producing a safe and idiomatic Rust translation of the input C program. The base prompt attempts to mitigate common issues observed in prior work on LLM-based code generation and translation, such as unimplemented functions [43] and safe Rust.

2) *Performance-Focused Prompt:* Building on the base prompt, the performance-focused prompt adds explicit instructions encouraging the model to optimize for runtime performance while maintaining safety and correctness. The goal of this prompt is to evaluate whether generating performant Rust code is as simple as instructing the LLM to do so, or whether more sophisticated techniques are required. In addition to all the instructions from the base prompt, this variant emphasizes that the generated code should be as performant as possible while still adhering to Rust’s safety guarantees.

3) *Multi-Variant Prompt:* Previous work [68] has shown that generating multiple variants of code allows the LLM to explore a wider search space, thus producing functionally equivalent variants that compile successfully. We extend this idea to the performance domain. Rather than generating a single Rust translation, we ask the model to produce multiple Rust transpilations of the same C program, with the goal of exploring a wider space of algorithmic, structural, and language-level choices that may affect runtime performance. Specifically, we instruct the model to generate five *performance-diverse* Rust implementations that preserve the same behavior as the original C code. We also instruct the model to accompany each variant with a brief comment explaining its strategy and expected performance implications, encouraging explicit performance reasoning.

4) *Summary-Guided Multi-Variant Prompt:* In the previous approaches, the model is provided with the original C source code. We use a two-stage approach to provide the coding agent with more freedom and encourage it to break away from C idioms. In the first stage, we prompt the agent to generate a high-level summary of the C program. Specifically, it extracts external libraries used, global definitions, functional blueprints (including parameters and notable I/O or memory behaviors),

and a macro-level architectural overview. This summary captures what the program does abstractly, anchors the critical definitions while avoiding any line-by-line description. In the second stage, we use this summary instead of the C source to guide the generation of multiple Rust variants. By decoupling the generation from the concrete C implementation, we aim to encourage more diverse algorithmic choices.

B. Oxivolve: Agentic Refinement

To maximize the performance of the transpiled Rust code, we introduce *Oxivolve*, an agentic code optimization pipeline. Moving beyond traditional search-based algorithms that operate within rigid, human-predefined spaces [45], [33], our framework leverages agents to navigate an open-ended optimization space of code configurations [82].

The optimization cycle operates via a single LLM agent fulfilling two sequential roles. First, acting as a *generator*, the agent populates the initial search space by producing 10 distinct, valid Rust variants, each exploring different algorithmic configurations and microarchitectural optimizations (such as loop unrolling, cache-friendly data layouts, and specialized library abstractions). Oxivolve executes the benchmark with selected inputs and feeds the execution metrics, including the average runtime and memory usage, back to the agent, which now acts as a *reflector*. Unlike traditional evolutionary algorithms [22], [35], Oxivolve’s agentic refinement mechanism does not provide an algebraic fitness function. Instead, Oxivolve relies on the coding agent itself to *reflect* on the execution data and perform a comparative analysis across the variants. This performance feedback helps the agent identify exactly which data structures or memory access patterns caused performance differentials. The agent then synthesizes a combined variant that merges the complementary strengths of the top-performing variants. This *generate* $\rightarrow$ *evaluate* $\rightarrow$ *recombine* approach allows Oxivolve to leverage agentic performance analysis capabilities while still grounding the choice of candidate variants in real, observed performance behaviors.

As depicted in Figure 1, the *Oxivolve* pipeline operates in three phases:

Phase 1 - Population Generation: The agent generates 10 functionally equivalent but structurally diverse Rust variants from the input C source [47], [67].

Phase 2 - Fitness: Each variant is compiled and executed in an isolated environment. We capture runtime and memory usage for each variant, yielding a performance profile. Instead of scalarizing these into a single fitness value, we present the full profiles to the agent in Phase 3. The agent then performs recombination directly, acting as an implicit Pareto selector.

Phase 3 - Agent-Driven Recombination (Crossover): The agent fuses the complementary strengths identified in Phase 2 into a single, syntactically valid implementation. This semantic recombination [58] fundamentally differs from traditional genetic crossover, which is typically a blind, syntactic operation (such as splicing abstract syntax trees) that frequently breaks

program semantics. By reasoning over execution profiles, the agent can understand *which* variant’s memory layout is efficient and *which* variant’s loop structure is fast, for example, and can fuse these insights into a correct, high-performance program. This iterative refinement through execution feedback aligns with recent findings on the self-correcting and reflective capabilities of large language models [55], [70], [82].

We incorporate a retry loop throughout the process. At any stage, if a variant fails compilation or execution, we prompt the agent with the resulting errors to fix the implementation.

IV. BENCHMARK DESIGN

We design our benchmarks to ensure high-fidelity performance evaluation. Both the C source and Rust variants are instrumented with high-resolution performance timers that record execution time only for the region of interest, excluding setup and teardown operations.

Deterministic Execution. For benchmarks relying on randomized inputs or operations, we enforced an identical `rand` seed across the C and Rust implementations to guarantee reproducibility. To achieve this parity, the Rust variants utilize the `libc` crate, providing a direct Foreign Function Interface (FFI) to ensure both language implementations invoke the exact same underlying C standard library functions.

Testing Infrastructure. To rigorously evaluate the Rust translation while preventing overfitting to specific software patterns, we split our evaluation into two distinct phases using a total of 66 test cases across the 10 benchmarks. First, for each benchmark, we use one large-scale, real-world dataset as the *public workload*. Its larger runtime is well suited for establishing baseline functional correctness and benchmarking optimization performance. Second, to assess the generalizability of the generated Rust code, we maintain a separate, *private validation set* comprised of diverse edge cases designed to test semantic preservation with respect to C. Both phases subject the transpiled variants to semantic equivalence checks, either via direct output matching or order-preserving checksum matching. This configuration mirrors the evaluation infrastructure employed in recent C-to-Rust translation work [15], [49], [73].

As shown by Anthropic [12] and Ahmed et al. [2], highly capable models frequently overfit to specific test cases or benchmark structures rather than solving the underlying task. Even if an agent is exposed to even the structural outline of a test case, it can synthesize an unfaithful implementation optimized exclusively for that known input shape. To prevent this, we dynamically load external files and utilize seed-based random generation. This approach forces the model to synthesize generalized, faithful translations rather than optimizing for a narrow, known input space.

V. EVALUATION

A. Benchmarks & AI Systems

Table I outlines the benchmarks used in our evaluation. We select benchmarks from prior C-to-Rust transpilation studies [25], [63], [86], [90], along with additional programs whose

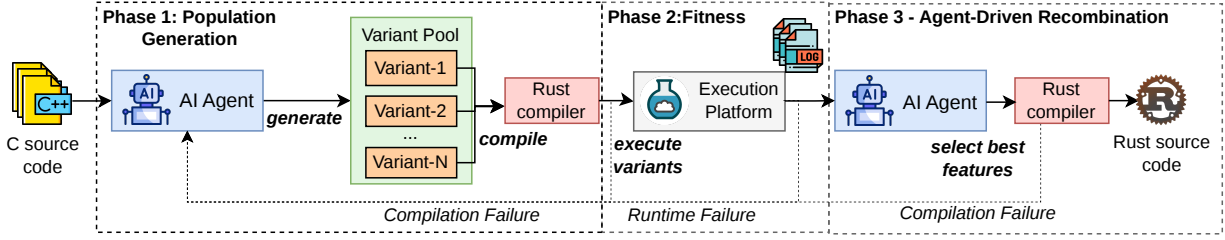


Fig. 1. Oxivolve pipeline. Note that each stage has correctness checks that are not shown.

Benchmark	Functionality	Workload
LibCSV [30]	CSV parsing and validation library	Validating 1.6 GB CSV (~1.06M rows, 19 cols) [78]
PageRank [64]	Iterative graph algorithm for node importance	LiveJournal graph (~4M nodes, ~35M edges) [81]
cJSON [29]	Lightweight JSON parsing and generation library	5,000 iterations over 809-entry Pokémon JSON [27]
RFFT [34]	Real Fast Fourier Transform (FFT) kernel	Forward FFT on an 8M-sample synthetic multi-frequency sine signal
LibBMP [20]	Bitmap (BMP) image library	Parsing 10,000 synthetic 100×100 images Writing an 8192×8192 image
K-Means [57], [53]	Unsupervised clustering via centroid assignment	1M points, $k=10$ clusters
Quicksort [7]	Sorting algorithm	Array of 100M elements
Matrix Mult. [69]	Dense matrix multiplication kernel	Two 2000×2000 matrices
SSSP [13]	Single-source shortest path on weighted graph	Graph of 30,000 nodes

TABLE I
BENCHMARKS AND WORKLOADS USED IN EVALUATION.

algorithmic diversity enables optimization through both Rust idioms and application-specific strategies. To contextualize the agent’s design decisions, we evaluate its performance against a *manual* Rust baseline developed by a software engineer with 7 years of general programming experience and 2 years of dedicated Rust expertise. This implementation allows us to observe whether the agent successfully utilizes standard abstractions, as a standard human developer would, or if it defaults to suboptimal C syntax [50], [89]. Across the 10 benchmarks, file size averages 351 LOC, ranging from 62 LOC (*Quicksort*) to 1,021 LOC (*cJSON*).

Our private validation sets and verification methodologies

test semantic preservation against C. For benchmarks with checksum-based validation like *Quicksort*, *cJSON*, *Matrix Multiplication*, and *LibBMP*, we accept rounding errors limited to the least significant digit as valid. If a variant failed any correctness check, it was *excluded* from our performance evaluation, i.e., only functionally correct transpilations were considered when analyzing runtime performance.

For evaluation, we use the *Gemini CLI* (v0.28.2) with *Gemini 3 Pro* as the underlying model, the *Codex CLI* (v0.106.0), with *GPT-5.2-Codex*, and *Claude Code* (v2.1.141) with *Claude Opus 4.6*. All agentic systems were used with their default configurations. In total, out of 410 variants that were generated across these systems, 354 passed our validation criteria, yielding a correctness rate of 86.3%.

B. Experimental Setup

To ensure a fair evaluation of the generated Rust variants against the C baselines, we conducted our experiments on the CloudLab [21] c6525-25g nodes in the Utah cluster. Each node is a Dell PowerEdge C6525 server equipped with AMD EPYC 7302P processors. All C benchmarks were compiled with the `clang-14.0.0` compiler, and the Rust benchmarks with the `rustc-1.61.0` stable with the exception of the manual Rust implementation *cJSON* benchmark, which was built with `rustc-1.61.0` nightly, enabling only the labeled break feature¹. Both `clang-14.0.0` and `rustc-1.61.0` use the LLVM-14 backend to compile to `x86_64` assembly code, ensuring that both the C and Rust benchmarks are optimized by the same LLVM pipeline using the `-O3` optimization flag. For all benchmarks, we measure execution time and memory usage averaged across 30 *independent runs*, preceding them with 5 *warm-up runs* to mitigate caching effects. We impose a five-minute timeout per execution test for each transpilation. Since our C benchmarks complete within seconds, this limit serves as a conservative upper bound to identify processes experiencing significant performance degradation. Overall, results are stable and suitable for comparison: average relative standard deviations are 0.86% for execution time and 0.81% for memory usage. Isolated worst-case variances in certain *LibCSV* benchmark variants reached 9.89% (execution time) and 18.68% (memory usage), but these outliers do not undermine the findings.

¹The labeled break feature was stabilized in `rustc-1.65.0`

Method	LibCSV	PageRank	cJSON	RFFT	LibBMP (P.)	LibBMP (W.)	K-Means	Quicksort	Matrix Mult.	SSSP
Baselines										
C	8.66 [1.18]	1.94 [326.92]	9.65 [3.54]	2.21 [129.50]	0.63 [1.18]	3.25 [193.56]	2.96 [31.85]	9.92 [382.73]	12.67 [47.37]	1.44 [9085.71]
Manual	9.36 [0.99]	2.67 [489.42]	9.90 [3.94]	2.11 [129.95]	1.49 [2.71]	3.73 [193.94]	2.97 [32.25]	10.83 [383.14]	2.12 [63.22]	2.51 [16145.70]
Gemini Agent										
Base	7.71 [1.03]	2.67 [326.98]	11.88 [3.78]	2.82 [129.95]	1.44 [1.00]	3.51 [194.20]	Failed [Failed]	10.41 [383.12]	18.35 [47.76]	2.06 [9091.34]
Perf.	9.16 [1.00]	2.88 [489.42]	9.03 [4.24]	2.76 [129.95]	1.37 [1.00]	Failed [Failed]	1.47 [32.25]	10.35 [383.13]	13.83 [47.49]	2.51 [16151.20]
M.V.	Failed [Failed]	1.70 [327.05]	6.86 [3.38]	0.48 [129.94]	0.20 [0.99]	Failed [Failed]	Failed [Failed]	10.47 [383.13]	6.27 [62.86]	0.86 [7361.60]
S.G.M.V.	1.79 [1574.23]	2.43 [1229.71]	Failed [Failed]	0.43 [129.96]	0.36 [0.99]	Failed [Failed]	Failed [Failed]	10.05 [383.13]	2.13 [63.11]	2.58 [10304.00]
Oxivolve	Failed [Failed]	1.87 [326.96]	8.16 [3.38]	2.03 [129.94]	0.25 [1.00]	3.23 [193.94]	1.22 [32.26]	10.27 [383.14]	2.11 [47.59]	0.48 [3837.66]
Codex Agent										
Base	8.31 [1.00]	Timed out [Timed out]	13.09 [4.29]	2.95 [129.94]	1.26 [0.99]	3.50 [194.20]	2.98 [32.27]	10.91 [383.15]	18.29 [47.70]	2.06 [9091.35]
Perf.	12.80 [0.99]	Timed out [Timed out]	9.62 [4.01]	2.77 [129.97]	0.44 [0.99]	3.55 [194.18]	2.95 [32.25]	11.02 [383.13]	2.10 [62.75]	2.53 [16151.50]
M.V.	10.09 [0.99]	Timed out [Timed out]	9.98 [4.29]	2.48 [194.49]	0.22 [0.99]	3.87 [193.93]	1.49 [32.27]	9.83 [383.14]	6.26 [62.84]	0.96 [15452.96]
S.G.M.V.	Failed [Failed]	2.00 [1925.15]	9.87 [4.03]	Failed [Failed]	0.24 [0.99]	Failed [Failed]	1.49 [32.25]	10.80 [383.13]	2.09 [62.77]	1.43 [7353.57]
Oxivolve	8.16 [0.99]	3.05 [489.67]	10.85 [3.63]	5.56 [129.96]	199.77 [0.99]	151.31 [194.19]	2.90 [28.39]	10.28 [383.11]	9.52 [62.96]	2.64 [16146.84]
Claude Agent										
Base	6.92 [0.99]	2.43 [326.97]	10.07 [3.77]	2.80 [130.22]	1.40 [0.99]	233.33 [194.18]	2.95 [32.24]	10.27 [383.13]	25.86 [47.73]	1.54 [9091.35]
Perf.	6.02 [2.12]	Timed out [Timed out]	9.46 [4.26]	2.77 [129.96]	0.40 [2.94]	3.49 [194.18]	2.96 [32.26]	10.40 [383.13]	7.53 [47.49]	2.50 [10304.00]
M.V.	5.38 [2.12]	2.27 [754.11]	4.05 [2.83]	4.93 [194.50]	0.25 [0.99]	3.21 [193.85]	1.40 [32.24]	9.94 [383.13]	6.20 [62.86]	0.61 [7363.71]
S.G.M.V.	Failed [Failed]	1.61 [1051.32]	7.75 [3.38]	0.43 [129.97]	0.22 [3.12]	3.25 [193.91]	1.48 [32.26]	9.95 [383.12]	6.74 [47.48]	0.64 [7355.86]
Oxivolve	1.80 [1574.23]	1.57 [1120.16]	3.57 [2.10]	1.31 [274.18]	0.23 [0.99]	3.26 [193.90]	1.51 [24.76]	9.69 [383.15]	1.72 [62.86]	0.60 [3837.43]

TABLE II
EXECUTION TIME (IN SECONDS) AND MEMORY USAGE [IN MB, SHOWN IN BRACKETS] ACROSS AGENTS (M.V.: MULTI-VARIANT, S.G.M.V.: SUMMARY-GUIDED MULTI-VARIANT). THE BEST RESULTS PER BENCHMARK **PER AGENT** ARE BOLD AND UNDERLINED.

C. High-Level Takeaways

Throughout our evaluation, we define an approach as “better” primarily through its execution time. While we monitor its memory consumption, we consider lower memory usage “nice-to-have”, since the evaluated benchmark sizes do not pressure the memory or exceed the hardware limitations of the CloudLab node.

Takeaway 1: Simple prompting strategies fail to close the performance gap. Both the Base Prompt and Performance-Focused Prompt produce code that largely regresses against the

C baseline (see Table II). The Base Prompt yields several severe slowdowns: Claude’s translation of *LibBMP (write)* runs at 233.33s against a C baseline of 3.25s, and *Matrix Multiplication* has runtimes of 25.86s (Claude) and 18.35s (Gemini) against a C baseline of 12.67s. Adding explicit performance instructions does not meaningfully correct this. Specifically, the Performance-Focused Prompt degrades *LibCSV* from 7.71s to 9.16s for Gemini, while Claude’s *K-Means* remains nearly unchanged at 2.96s vs. 2.95s. Both strategies also suffer from correctness failures and *PageRank* timeouts across Codex and

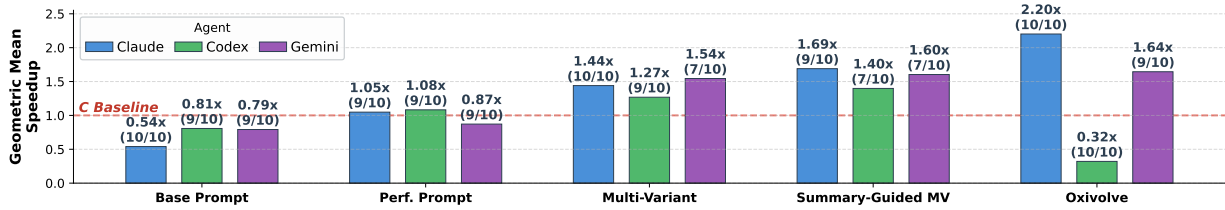


Fig. 2. Geometric Mean Speedup across all 10 Benchmarks (Test Pass Rate shown in parentheses). Failed benchmarks default to C speedup of 1.0 $\times$.

Claude. Furthermore, even a manual Rust implementation by an experienced developer (written without any explicit performance consideration) achieves near-parity (within 10%) of C on 6 out of 10 benchmarks, which simple performance prompting barely meets. This confirms that generic directives are insufficient and justifies the need for more sophisticated techniques.

Takeaway 2: Multi-variant generation drives performance improvement. Generating a diverse population of implementations and selecting the fastest produces consistent, substantial gains across all three agents. Our Multi-Variant (M.V.) strategy on Gemini reduces *PageRank* from 2.67s to 1.70s and *LibBMP (parse)* from 1.37s to 0.20s. Claude reduces *cJSON* from 10.07s to 4.05s and *SSSP* from 1.54s to 0.61s. Codex achieves its fastest *SSSP* at 0.96s. In fact, this strategy generates the best-performing, fully correct implementation in 5 out of 10 benchmarks for Codex, 3 out of 10 for Gemini, and 2 out of 10 for Claude. However, certain failures like Gemini on *LibCSV* and *K-Means* still persist. The broader takeaway is that exploring a wider space of algorithmic and language-level choices yields performant implementations that a single-shot prompt cannot generate, which persists even in the Summary-Guided Multi-Variant (S.G.M.V.) and Oxivolve strategies. This is the core finding that motivates the rest of the paper.

Takeaway 3: Summary-Guided Multi-Variant improves performance at the cost of correctness. Abstracting the C source into a high-level summary before generation produces some of the fastest individual results: Gemini reaches 1.79s on *LibCSV*, and both Gemini and Claude achieve 0.43s on *RFFT*, compared to 2.76s and 2.77s respectively under the Performance-Focused Prompt. Across our evaluation, this strategy generates the best-performing, fully correct implementation in 3 out of 10 benchmarks for Codex, 3 out of 10 for Gemini, and 2 out of 10 for Claude. However, this performance comes with the trade-off of an overall reduction in correctness. The approach fails on 3 of 10 benchmarks for both Codex and Gemini, and 1 of 10 for Claude, with failures that persisted even after an average of ~ 20 lines of manual intervention per variant because the summaries strip away details necessary for semantic equivalence. Thus, Summary-Guided Multi-Variant has a correctness-performance tradeoff and cannot be used without human intervention.

Takeaway 4: Oxivolve resolves the correctness bottleneck while preserving performance gains. Oxivolve achieves the

fastest runtimes on 6 of 10 benchmarks for Claude, including *Matrix Multiplication*, *PageRank*, and *SSSP*, with zero test failures. Gemini similarly attains its best results on 4 out of 10 benchmarks, including *K-Means* and *SSSP*. Simultaneously, Oxivolve also results in the lowest memory usage on 6 out of 10 benchmarks for Codex, 5 for Gemini, and 4 for Claude. The exception to the runtime improvements is Codex, which does not perform well under Oxivolve. As we discuss later, we consider this a consequence of its variant population lacking sufficient diversity for the recombination phase to operate on.

Geometric Mean Speedup: To provide a high-level summary of the overall performance trends, we calculate the geometric mean speedup across all 10 benchmarks over the C baseline. To ensure a fair comparison, benchmarks failing validation are conservatively assigned the C baseline speedup of 1.0 $\times$. This effectively grants them the benefit of the doubt that they would perform on par with C if they were correctly implemented [72], [83]. This fallback is fair specifically because Oxivolve has the least number of failing benchmarks and therefore rarely relies on it. The other steering mechanisms fail more often, and thus, the fallback has more influence on their geomean than for Oxivolve.

As seen in Figure 2, Oxivolve achieves geometric mean speedups of 2.20 $\times$ (Claude), 0.32 $\times$ (Codex), and 1.64 $\times$ (Gemini) across all 10 benchmarks. Therefore, these results demonstrate that our feedback-driven recombination holds promise for systematic performance optimization during transpilation while not sacrificing correctness. We discuss Codex’s performance next.

Takeaway 5: Variant diversity drives Oxivolve’s efficacy. To isolate the effects of generation and recombination, we performed cross-model recombination, i.e., each agent recombined the 10 variants produced by the other. When Gemini serves as the generator, both pipelines achieve high speedups (1.64 $\times$ for Gemini’s own pipeline; 1.57 $\times$ for Codex recombining Gemini’s variants). Conversely, when Codex generates the initial pool, performance remains constrained regardless of the recombining agent (0.32 $\times$ for Codex; 0.39 $\times$ for Gemini recombining Codex’s variants). Root-cause analysis revealed that Codex’s *LibBMP* (both *parse* and *write*) and *RFFT* variants were almost identical and thus, provided no optimization signal. This shows that the diversity of the initial population is more important than the recombination capability of the agent.

D. Performance Optimization Categories

In this section, we refer to the optimizations made by the Oxivolve pipeline due to the availability of rich explanations from the agents. While many optimization techniques are benchmark-specific, we observed that they can be abstracted into several overarching categories.

I/O and Buffering Operations. File I/O workloads aim to bridge the massive performance gap between the CPU and the storage disk. To minimize expensive system calls and context switches, I/O optimizations such as buffering can be employed. Instead of reading or writing byte-by-byte, data can be collected and processed in large blocks to take advantage of spatial locality [42]. Once the data is loaded, zero-copy techniques can be applied to scan raw bytes directly, bypassing the overhead of unnecessary string allocations and copy operations. Furthermore, pre-allocation, i.e., allocating the precise capacity required for a file or string upfront, eliminates the overhead of dynamic buffer resizing, which would otherwise force the program to allocate new memory and copy existing data *during* processing, increasing the operation latency.

These techniques were applied by the agents for the *K-Means*, *LibCSV*, *cJSON*, *LibBMP (parse)* and *LibBMP (write)* benchmarks. Codex employed pre-allocation for internal data structures for *LibCSV*, while all agents utilized pre-allocation for *K-Means*. Gemini took advantage of the `COW` (Clone-on-Write) smart pointer and reusable buffers in *cJSON* implementations. For *LibBMP*, the Claude and Gemini agents focused primarily on reducing I/O system calls in combination with similar pre-allocation and buffering techniques.

Loop and Branch Optimizations. The Rust compiler automatically inserts bounds-checks for array accesses. While these checks ensure safety, they introduce conditional branches that can hamper performance by stalling the CPU pipeline [74]. We can elide these checks by leveraging Rust’s iterators and slice patterns. These abstractions often compile down to direct, bounds-check-free pointer arithmetic. Manual loop unrolling, i.e., executing multiple iterations of a loop body within a single step can also drastically reduce the overhead of evaluating the loop termination condition. Loop and branch optimizations were applied to the *PageRank*, *LibBMP (parse)*, and *Matrix Multiplication* benchmarks, while branch hoisting was utilized by Gemini in both the *RFFT* and *LibBMP (write)* benchmarks.

Algorithmic Optimizations. Instead of merely optimizing specific operations or code blocks, often changing the algorithmic structure can achieve better performance. Specifically, naive recursive implementations can be replaced with approaches that minimize function call overhead and avoid deep recursion. For example, naive Merge Sort incurs significant overhead by maintaining a stack of partitions. It can be replaced by an iterative bottom-up approach, which eliminates the cost of recursive calls and improves cache locality. Standard graph processing algorithms can be optimized to prune search spaces and eliminate redundant evaluations. For

instance, PageRank performance improves significantly when updates are restricted to modified subgraphs, bypassing the need for full global recomputation [19].

Algorithmic optimizations were applied to the *Quicksort*, *SSSP*, and *K-means* benchmarks. For *Quicksort*, all agents implemented hybrid approaches to the algorithm, switching to Insertion Sort for smaller partitions accompanied by other algorithmic differences such as median-of-three pivot selection and tail-call optimization. *SSSP* saw implementations of the Shortest Path Faster Algorithm [88] with Gemini and Claude, while *K-Means* saw fused operations, early convergence, and the use of squared instead of `sqrt` distance.

Memory Access Patterns. Memory access patterns determine how efficiently the CPU can retrieve data from memory. Since fetching data directly from main memory is significantly slower than accessing the CPU cache, processors prioritize storing frequently used data in L1 and L2 hardware caches to minimize latency. A different data layout can maximize cache hits. For example, using Structure of Arrays (SoA) instead of Array of Structures (AoS) to represent a collection of records often yields better performance by packing related fields into contiguous memory, allowing a single cache line to serve multiple iterations.

The agents targeted data layout for the *K-Means*, *Matrix Multiplication*, and *RFFT* benchmarks. *K-Means* saw different attempts at SoA/AoS between agents, Claude utilized a flat vector with interleaved real and imaginary entries for *RFFT*, and all agents similarly used a 1D flat vector for *Matrix Multiplication* to promote better cache locality.

E. Case Study: K-Means Clustering

In this section, we explore the optimizations applied by the agents in the Oxivolve pipeline on the *K-Means Clustering* algorithm. The K-Means algorithm initializes an estimate of centroid positions that it refines iteratively until convergence for N data points and K centroids. Since an implementation of this algorithm combines arithmetic operations (updating centroids) with memory operations (iterating through the data points), it has a diverse search space of optimization opportunities, ranging from better data structures for storing points, to simplification of the arithmetic and the algorithm itself, to reduction of memory pressure. Therefore, it is an ideal choice for evaluating the efficacy of Oxivolve, and gaining an insight into the tradeoffs it made.

C Baseline Implementation. This implementation followed what is traditionally known as Lloyd’s algorithm [53]. For each of the N points, it computed the distance to all K centroids using the `sqrt` function, and performed cluster assignment and centroid updates in two separate loops. Data points were stored in an Array-of-Structures (AoS) layout, with each `Point` struct occupying 32 bytes (`x`, `y`, `cluster_id`, `cluster_distance`). It had a runtime of 2.96 seconds and a memory usage of 31.85 MB.

Optimization Search Space. A wide range of optimizations is possible for K-Means. We outline those observed in Oxivolve’s initial 10 variants below.

- **Memory layout:** Storing points in SoA (flat arrays) can reduce L1D cache misses by eliminating alignment padding within structs [3]. Conversely, if an AoS struct fits within a 64-byte cache line, it can exploit spatial locality, since fields like `x` and `y` are accessed together.
- **Distance computation:** Comparing squared distances rather than calling `sqrt()`. Since squaring preserves ordering on non-negative values, only relative ordering matters here, and squared distance is cheaper to compute.
- **Pass fusion:** The centroid update and cluster assignment loops can be combined into a single pass, reducing the number of memory loads by 50%.
- **Buffer pre-allocation & reuse:** Intermediate buffers (`sum_x`, `sum_y`) can be pre-allocated and reused across iterations, eliminating repeated heap allocations.
- **Early termination:** The algorithm can break out of the iteration loop early if no cluster assignments change.
- **Reduced struct size:** Redundant fields in the `Point` struct can be removed to lower memory traffic.
- **Alternative Algorithm:** Elkan’s algorithm can replace Lloyd’s. It uses triangle inequality to avoid unnecessary distance calculations during cluster assignment [23].

Oxivolve Recombinations. The recombined variants by each agent applied different combinations of the aforementioned optimizations. Claude’s variant incorporated early termination, pre-allocated, reused buffers, squared distance computation, and an AoS layout with a reduced struct size (removing the redundant `cluster_distance` field). Codex implemented an SoA layout, storing `x` and `y` points in flat `f64` vectors. Gemini combined an AoS layout with pass fusion, squared distance computation, and pre-allocated and reused buffers. These variants achieved speedups over the C baseline of $2.43\times$ for Gemini, $1.96\times$ for Claude, and $1.02\times$ for Codex.

We noticed that Claude’s initial variant population also explored Elkan’s algorithm. A pitfall of this algorithm is that it requires storing lower and upper bounds on point-to-centroid distances, which, for 1 million points, inflated memory usage to over 100 MB, and resulted in a runtime of 4.45 seconds. Claude was able to identify this pitfall through the feedback and avoid recombining features from this variant into the final implementation, even though this is a well-known optimization. In fact, both Codex and Claude reduced memory usage relative to every other steering mechanism, while Gemini’s memory usage remained the same. Therefore, in this benchmark, the richness of the runtime and memory feedback enabled the agents to discover implementations that minimized both objectives rather than optimizing runtime in isolation. Overall, the Oxivolve pipeline can, thus, explore diverse implementations and optimization opportunities, and its recombination phase can effectively use both signals and generate a performant recombined variant.

Model	Microarch.			Inst.-level			Total
	✓	~	×	✓	~	×	
Gemini	5	1	5	5	3	0	19
Codex	2	6	0	0	2	1	11
Claude	4	1	0	1	0	0	6
Total	11	8	5	6	5	1	36

TABLE III

PER-MODEL OPTIMIZATION CLAIM OUTCOMES, SPLIT BY CLAIM CATEGORY (✓ CORRECT, ~ INCONCLUSIVE (<1% DIFF.), × INCORRECT)

F. Validating Agents’ Microarchitectural Optimization Claims

To evaluate the agents’ effectiveness at making microarchitectural optimizations, we performed a *post-hoc validation*, systematically comparing the explanations they produced against empirical hardware profiling results. We verified these claims by using the Linux `perf` tool to capture granular hardware performance counters, such as branch misses, L1 data (L1D) cache misses and instruction counts [16]. The empirical validation results for Gemini, Codex, and Claude are detailed in Table III. Out of the microarchitectural claims evaluated, Gemini was fully correct on 10 of 19 claims, Codex on 2 of 11, and Claude on 5 of 6.

We find that agents are generally effective at predicting instruction-level outcomes, but frequently struggle to anticipate complex hardware-software interactions, such as cache hierarchies, i.e., when optimizing branching behaviors or compiler-level abstractions where the mapping to instructions is direct, agents’ intuitions align well with hardware execution. For example, Claude accurately predicted that replacing a lookup table with direct byte comparison (`==`) in *LibCSV* would yield more predictable branches, confirmed by a lower branch miss rate (2.90% vs. 3.46%) and higher IPC (2.08 vs. 1.87). Similarly, Gemini correctly identified that `.chunks_exact` in *LibBMP (parse)* would elide array bounds checks, reducing executed instructions by 39.5%, and that idiomatic `match` statements and the `matches!()` macro in *LibCSV* and *cJSON* reduce branch predictor misses and branch instructions.

On the other hand, agents frequently mispredicted outcomes governed by cache hierarchy dynamics. Gemini recommended an Array of Structs (AoS) layout for *K-Means* to reduce cache thrashing, yet AoS incurred 12% *more* L1D cache misses than SoA, as the wider struct layout conflicted with cache line boundaries. Codex’s *LibBMP (write)* optimization similarly backfired: row borrowing reduced L1D misses but simultaneously inflated L1D loads, resulting in slower execution. Notably, Codex struggled to identify effective optimizations. While it correctly predicted cache-locality improvements such as SoA layout and loop tiling, it consistently overestimated the benefit of replacing iterators with manual indexing across *LibCSV*, *K-Means*, and *SSSP*.

Thus, although agents can produce optimized code, they often provide misguided justifications for their performance gains. This aligns with recent literature which demonstrates that LLMs often arrive at successful outcomes while gen-

erating plausible, yet incorrect, post-hoc rationalizations for their decisions [76], [46]. This inconsistency between the agents’ expectations and real-world performance results is why *Oxivolve* relies on actual hardware execution metrics rather than model intuition to generate the final implementation.

G. Qualitative Analysis of Variant Failure Modes

We conducted a root-cause analysis of all test failures. *LibBMP (parse)*, *Quicksort*, *Matrix Mult.*, and *SSSP* passed all tests across all variants. Of the failing cases, *Oxivolve* produced the fewest failures (1), followed by Base (2), Perf.-Focused Prompt (3), Multi-Variant (4), and Summary-Guided (7). Our analysis shows that many of the failing implementations are still very close to being correct, with failures occurring due to minor translation errors rather than fundamental algorithmic mistakes. We categorize them into three patterns:

Type-System Mismatches. In *PageRank*, failed variants suffer from a signed-to-unsigned integer bug: a signed subtraction (`row_ptr[i+1] - row_ptr[i]`) that safely evaluates to a negative value in C triggers an integer underflow to `usize::MAX` in Rust, hanging the program indefinitely.

Initialization and Configuration Errors. In *K-Means*, failing variants initialize centroids randomly or at (0,0) rather than matching the C reference, yielding divergent final coordinates despite algorithmically sound clustering logic. Similarly, *LibBMP (write)* failures arise from spatial flips or RGB channel misalignment (e.g., RGB) and *RFFT* produces values scaled incorrectly by a factor of n or $1/n$ due to normalization context lost in the summary.

Specification Misinterpretation. In *LibCSV*, one variant misinterprets the validation task entirely (counting rows instead of parsing), while others improperly handle quoted fields. In *cJSON*, the failing variants miscount boolean values and neglect to strip backslashes during string extraction.

VI. DISCUSSION

Summary-Guided Multi-Variant Approach. While this approach produced performant Rust implementations, it required substantial manual intervention: an average of approximately 20 lines of code modifications per variant to ensure semantic equivalence with the original C code. High-level summaries inherently strip away granular requirements (explicit `rand` seeds, timing hooks, file parsing logic) that guarantee equivalence with the original C source. This limitation closely mirrors the challenges in aligning abstract documentation with its equivalent code [65]. After these interventions, Claude passed 9/10 benchmarks against the test suite, while Gemini and Codex both passed only 7/10.

Cost. The *Oxivolve* pipeline increases computational cost, requiring an average of approximately 10 minutes to execute to completion (including generation, execution, refinement, and re-execution). Agentic frameworks in headless mode do not reliably report token usage at runtime. We extracted metrics post-hoc via the Claude Admin API, yielding a total of 3M input and 80K output tokens across the full evaluation.

Optimizing Rust vs. Optimizing C. The performance gains achieved by *Oxivolve* could be realized by optimizing the original C baselines. However, some optimizations, such as bounds-checking elision, depend on language features, and therefore do not apply to C, which lacks built-in memory safety and bounds checks. Conversely, strategies such as utilizing Copy-on-Write (COW) smart pointers are native to Rust but would require substantial manual engineering to safely replicate in C. As a whole, we view these optimizations as the agent rearchitecting the code for Rust-specific performance while simultaneously streamlining underlying algorithmic inefficiencies in the original C source.

VII. THREATS TO VALIDITY

Internal validity. *Oxivolve*’s agentic workflows are inherently non-deterministic due to multi-step reasoning and iterative tool-calls, so the “combined” variant may vary across runs. To control measurement noise, we used isolated CloudLab nodes and Region of Interest (ROI) annotations to exclude initialization overhead. While hardware-specific features such as branch predictors and cache architectures affect optimization efficacy, *Oxivolve* is environment-agnostic: it relies on real-time performance feedback, making it applicable across diverse hardware and compiler toolchains.

External validity. Our evaluation used a curated suite of ten widely-used programs. Though algorithmically diverse, these are single-file workloads that may not capture the complexity of large-scale, multi-file C codebases; we view single-file optimization as a precursor to such migration and leave architectural scaling to future work. Finally, our study targeted Gemini 3 Pro, GPT-5.2-Codex, and Claude Opus 4.6, so findings may not generalize to other models.

VIII. RELATED WORK

C-to-Rust Transpilation. Rewriting of C to Rust is labor-intensive, motivating the use of automated transpilers. Early tools like C2Rust [1] generate “unsafe” Rust using raw pointers. Subsequent frameworks, such as LAERTES [25], [24], CROWN [86], and CRustS [51], attempt to refactor this into safe Rust using static analysis and AST transformations. More recently, LLM-based approaches leverage ownership analysis, slicing, and data-flow analysis to produce more idiomatic code [92], [37], [8], [71], [63]. Others use hybrid strategies combining LLMs with peephole optimizations (PR² [31]), rule-based transformations (Oxidizer [85]), or context pre-processing (RustAssure [6]). Despite generating safer code, both traditional and LLM-based methods translate at the line or function level, preserving C’s definitions and control flow rather than actively rethinking the program for performance.

AI Code Performance. Despite functional correctness, AI-generated code often suffers from performance regressions due to suboptimal algorithmic choices and underutilization of language features. Studies on major LLMs and GitHub Copilot highlight significant gaps compared to human-authored

code [48], [79]. Frameworks like DPE [52] provide stress-testing for these inefficiencies, while PerfOrch [10] and PERFCODEGEN [66] attempt to bridge the gap through dynamic model routing and execution-guided refinement. Building on these insights, Oxivolve employs execution-guided refinement specifically for performant C-to-Rust transpilation.

IX. CONCLUSION

We showed that performance-aware C-to-Rust transpilation benefits from systematic exploration of diverse implementation strategies rather than single-shot prompting. Our multi-variant framework and agentic evolution loop Oxivolve broadly outperformed baselines, demonstrating that treating runtime performance as a first-class objective can meaningfully help refine and optimize agent-generated Rust.

X. DATA AVAILABILITY STATEMENT

To prevent premature disclosure, our source code and dataset are provided for review via a private link (<https://figshare.com/s/f61bc2e98ca59408036a?file=65849589>). Upon acceptance, the camera-ready revision will be updated to include a permanent, public DOI.

REFERENCES

- [1] “C2rust,” 2024. [Online]. Available: <https://github.com/immunant/c2rust>
- [2] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel, “Investigating test overfitting on swe-bench,” 2026. [Online]. Available: <https://arxiv.org/abs/2511.16858>
- [3] V. Alfred, S. Monica, S. Ravi, U. Jeffrey D *et al.*, *Compilers Principles, Techniques*. Pearson, 2007.
- [4] S. Ashish and B. Jack, “Matrix-multiplication/c_matrix_multiplication.c,” https://github.com/makeuseofcode/Matrix-Multiplication/blob/main/c_matrix_multiplication.c, 2022, commit: be748b1.
- [5] A. Avizienis, “The n-version approach to fault-tolerant software,” *IEEE Transactions on software engineering*, no. 12, pp. 1491–1501, 1985.
- [6] Y. Bai and T. Palit, “Rustassure: Differential symbolic testing for llm-transpiled c-to-rust code,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 534–546.
- [7] G. S. Brodal, R. Fagerberg, and G. Moruz, “On the adaptiveness of quicksort,” *Journal of Experimental Algorithmics (JEA)*, vol. 12, pp. 1–20, 2008.
- [8] X. Cai, J. Liu, X. Huang, Y. Yu, H. Wu, C. Li, B. Wang, I. N. B. Yusuf, and L. Jiang, “Rustmap: Towards project-scale c-to-rust migration via program analysis and llm,” *arXiv preprint arXiv:2503.17741*, 2025.
- [9] N. Carlini. (2026, Feb) Building a c compiler with a team of parallel claudes. Anthropic. [Online]. Available: <https://www.anthropic.com/engineering/building-c-compiler>
- [10] H. Chen, Z. Qi, H. Li, H. Chen, J. Chen, K. Peng, I. K. Kim, K. H. Lee, and S. He, “Beyond single llms: Enhanced code generation via multi-stage performance-guided llm orchestration,” *arXiv preprint arXiv:2510.01379*, 2025.
- [11] CoffeeBeforeArch, “mmul: Serial and parallel implementations of matrix multiplication,” <https://github.com/CoffeeBeforeArch/mmul>, 2021, commit: c624ef7.
- [12] R. Coleman, “Eval awareness in Claude Opus 4.6’s BrowseComp performance,” <https://www.anthropic.com/engineering/eval-awareness-browsecomp>, 2026, engineering at Anthropic. Accessed: 2026-03-13.
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [14] B. Cox, D. Evans, A. Filipi, J. Rowanhill, W. Hu, J. Davidson, J. Knight, A. Nguyen-Tuong, and J. Hiser, “N-variant systems: A secretless framework for security through diversity,” in *USENIX Security Symposium*, vol. 114, 2006, p. 114.
- [15] DARPA TRACTOR Program, “TRACTOR Public Test Corpus,” <https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus>, 2024.
- [16] A. C. De Melo, “The new linux ‘perf’ tools,” in *Slides from Linux Kongress*, vol. 18, no. 1, 2010.
- [17] K. Deb, “Multi-objective optimisation using evolutionary algorithms: an introduction,” in *Multi-objective evolutionary optimisation for product design and manufacturing*. Springer, 2011, pp. 3–34.
- [18] S. Dehghan, T. Sun, T. Wu, Z. Li, and R. Jabbarvand, “Translating large-scale c repositories to idiomatic rust,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.20617>
- [19] P. Desikan, N. Pathak, J. Srivastava, and V. Kumar, “Incremental page rank computation on evolving graphs,” in *Special interest tracks and posters of the 14th International Conference on World Wide Web*, 2005, pp. 1094–1095.
- [20] M. V. Dickmann, “libbmp: A simple bitmap (bmp) library,” <https://github.com/marc-q/libbmp>, 2017, accessed: 2026-06-03.
- [21] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, “The design and operation of CloudLab,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Jul. 2019, pp. 1–14. [Online]. Available: <https://www.flux.utah.edu/paper/duplyakin-atc19>
- [22] A. E. Eiben and J. E. Smith, *Introduction to evolutionary computing*. Springer, 2015.
- [23] C. Elkan, “Using the triangle inequality to accelerate k-means,” in *Proceedings of the 20th international conference on Machine Learning (ICML-03)*, 2003, pp. 147–153.
- [24] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, “Aliasing limits on translating c to safe rust,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 551–579, 2023.
- [25] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, “Translating c to safer rust,” vol. 5, pp. 1–29, publisher: ACM New York, NY, USA.
- [26] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, “Towards translating real-world code with llms: A study of translating to rust,” *arXiv preprint arXiv:2405.11514*, 2024.
- [27] Fanzeyi, “pokemon.json: Pokémon dataset in JSON format,” <https://github.com/fanzeyi/pokemon.json>, 2023.
- [28] S. G. Ficici and J. B. Pollack, “Pareto optimality in coevolutionary learning,” in *European Conference on Artificial Life*. Springer, 2001, pp. 316–325.
- [29] D. Gamble, “cJSON,” <https://github.com/DaveGamble/cJSON>, 2023.
- [30] R. Gamble, “libcsv: Fast and flexible csv library written in pure ansi c that can read and write csv data,” accessed: 2025-09-18. [Online]. Available: <https://github.com/rgamble/libcsv>
- [31] Y. Gao, C. Wang, P. Huang, X. Liu, M. Zheng, and X. Zhang, “Pr2: Peephole raw pointer rewriting with llms for translating c to safer rust,” *arXiv preprint arXiv:2505.04852*, 2025.
- [32] W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, “Large language model evaluation for high-performance computing software development,” *Concurrency and Computation: Practice and Experience*, vol. 36, no. 26, p. e8269, 2024.
- [33] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, 1st ed. USA: Addison-Wesley Longman Publishing Co., Inc., 1989.
- [34] grego, “rfft.h: Reasonably fast fourier transform in a single header for c and c++,” <https://github.com/grego/rfft.h>, 2023, accessed: 2026-06-03.
- [35] M. Harman, S. A. Mansouri, and Y. Zhang, “Search-based software engineering: Trends, techniques and applications,” *ACM Computing Surveys (CSUR)*, vol. 45, no. 1, pp. 1–61, 2012.
- [36] C. Hong, S. Bhatia, A. Cheung, and S. Shao, “Autocomp: Llm-driven code optimization for tensor accelerators,” in *Machine Learning for Computer Architecture and Systems 2025*, 2025.
- [37] J. Hong and S. Ryu, “Type-migrating c-to-rust translation using a large language model,” vol. 30, no. 1, p. 3, publisher: Springer.
- [38] —, “Don’t write, but return: Replacing output parameters with algebraic data types in c-to-rust translation,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 716–740, 2024.
- [39] —, “To tag, or not to tag: Translating c’s unions to rust’s tagged unions,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 40–52.
- [40] —, “Forcrat: Automatic i/o api translation from c to rust via origin and capability analysis,” *arXiv preprint arXiv:2506.01427*, 2025.
- [41] T. Jackson, B. Salamat, G. Wagner, C. Wimmer, and M. Franz, “On the effectiveness of multi-variant program execution for vulnerability

- detection and prevention,” in *Proceedings of the 6th International Workshop on Security Measurements and Metrics*, 2010, pp. 1–8.
- [42] A. Jacobs, “The pathologies of big data,” *Communications of the ACM*, vol. 52, no. 8, pp. 36–44, 2009.
- [43] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” *arXiv preprint arXiv:2310.06770*, 2023.
- [44] A. Kling. (2026, Feb) Ladybird adopts rust, with help from ai. Ladybird. [Online]. Available: <https://ladybird.org/posts/adopting-rust/>
- [45] J. R. Koza, “Genetic programming as a means for programming computers by natural selection,” *Statistics and computing*, vol. 4, no. 2, pp. 87–112, 1994.
- [46] T. Lanham, A. Chen, A. Radhakrishnan, B. Steiner, C. Denison, D. Hernandez, D. Li, E. Durmus, E. Hubinger, J. Kernion *et al.*, “Measuring faithfulness in chain-of-thought reasoning,” *arXiv preprint arXiv:2307.13702*, 2023.
- [47] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh, and K. O. Stanley, “Evolution through large models,” in *Handbook of evolutionary machine learning*. Springer, 2023, pp. 331–366.
- [48] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang, “Assessing the performance of ai-generated code: A case study on github copilot,” in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2024, pp. 216–227.
- [49] T. Li, R. Li, B. Wang, B. Paulsen, U. Mathur, and P. Saxena, “Adversarial agent collaboration for c to rust translation,” *arXiv preprint arXiv:2510.03879*, 2025.
- [50] S. A. Licorish, A. Bajpai, C. Arora, F. Wang, and K. Tantiathamthavorn, “Comparing human and llm generated code: The jury is still out!” *arXiv preprint arXiv:2501.16857*, 2025.
- [51] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, “In rust we trust: a transpiler from unsafe c to safer rust,” in *Proceedings of the ACM/IEEE 44th international conference on software engineering: companion proceedings*, pp. 354–355.
- [52] J. Liu, S. Xie, J. Wang, Y. Wei, Y. Ding, and L. Zhang, “Evaluating language models for efficient code generation,” *arXiv preprint arXiv:2408.06450*, 2024.
- [53] S. Lloyd, “Least squares quantization in pcm,” *IEEE transactions on information theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [54] I. Lozano and D. Maier, “Deploying rust in existing firmware codebases,” <https://security.googleblog.com/2024/09/deploying-rust-in-existing-firmware.html>, Sep. 2024, google Security Blog.
- [55] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [56] B. Marker, F. G. Van Zee, K. Goto, G. Quintana-Ortí, and R. A. Van De Geijn, “Toward scalable matrix multiply on multithreaded architectures,” in *European Conference on Parallel Processing*. Springer, 2007, pp. 748–757.
- [57] J. B. McQueen, “Some methods of classification and analysis of multivariate observations,” in *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.*, 1967, pp. 281–297.
- [58] E. Meyerson, M. J. Nelson, H. Bradley, A. Gaier, A. Moradi, A. K. Hoover, and J. Lehman, “Language model crossover: Variation through few-shot prompting,” *ACM Transactions on Evolutionary Learning*, vol. 4, no. 4, pp. 1–40, 2024.
- [59] Mozilla, “Put your trust in rust — shipping now in firefox,” <https://blog.mozilla.org/en/firefox/put-trust-rust-shipping-now-firefox/>, Mar. 2017, accessed 2026-03-24.
- [60] N. Nader, P. Diehl, S. Brandt, and H. Kaiser, “Llm & hpc: Benchmarking deepseek’s performance in high-performance computing tasks,” in *International Conference on High Performance Computing*. Springer, 2025, pp. 626–638.
- [61] M. Nejjar, L. Zacharias, F. Stiehle, and I. Weber, “Llms for science: Usage for code generation and data analysis,” *Journal of Software: Evolution and Process*, vol. 37, no. 1, p. e2723, 2025.
- [62] A. Nistor, *Understanding, detecting, and repairing performance bugs*. University of Illinois at Urbana-Champaign, 2014.
- [63] V. Nitin, R. Krishna, L. L. d. Valle, and B. Ray, “C2saferust: Transforming c projects into safer rust with neurosymbolic techniques,” *arXiv preprint arXiv:2501.14257*, 2025.
- [64] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking: Bringing order to the web.” Stanford infolab, Tech. Rep., 1999.
- [65] K. Pai, P. Devanbu, and T. Ahmed, “Codocbench: A dataset for code-documentation alignment in software maintenance,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 451–455.
- [66] Y. Peng, A. D. Gotmare, M. R. Lyu, C. Xiong, S. Savarese, and D. Sahoo, “Perfcodegen: Improving performance of llm generated code with execution feedback,” in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 2025, pp. 1–13.
- [67] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi *et al.*, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.
- [68] J. Ron, D. Gaspar, J. Cabrera-Arteaga, B. Baudry, and M. Monperrus, “Galapagos: Automated n-version programming with llms,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [69] O. Schwartz and N. Vaknin, “Pebbling game and alternative basis for high performance matrix multiplication,” *SIAM Journal on Scientific Computing*, vol. 45, no. 6, pp. C277–C303, 2023.
- [70] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflection: Language agents with verbal reinforcement learning,” *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.
- [71] M. Shiraishi and T. Shinagawa, “Context-aware code segmentation for c-to-rust translation using large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.10506>
- [72] A. Shypula, A. Madaan, Y. Zeng, U. Alon, J. Gardner, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani, and A. Yazdanbakhsh, “Learning performance-improving code edits,” 2024. [Online]. Available: <https://arxiv.org/abs/2302.07867>
- [73] H. Sim, H. Cho, Y. Go, S. A. K. Zarkouei, Z. Fu, A. Shokri, and B. Ravindran, “Search-based multi-trajectory refinement for safe c-to-rust translation with large language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2505.15858>
- [74] J. E. Smith, “A study of branch prediction strategies,” in *25 years of the international symposia on Computer architecture (selected papers)*, 1998, pp. 202–215.
- [75] The Linux Kernel Documentation, “Rust,” <https://docs.kernel.org/rust/index.html>, 2026, accessed 2026-03-24.
- [76] M. Turpin, J. Michael, E. Perez, and S. Bowman, “Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 74 952–74 965, 2023.
- [77] F. Valencia-Clavijo, “Anchors in the machine: Behavioral and attributional evidence of anchoring bias in llms,” *arXiv preprint arXiv:2511.05766*, 2025.
- [78] L. L. Wang *et al.*, “CORD-19: The COVID-19 open research dataset,” in *Proceedings of the 1st Workshop on NLP for COVID-19 at ACL 2020*, K. Verspoor *et al.*, Eds. Online: Association for Computational Linguistics, Jul. 2020. [Online]. Available: <https://aclanthology.org/2020.nlpCOVID19-acl.1/>
- [79] L. Wang, C. Shi, S. Du, Y. Tao, Y. Shen, H. Zheng, and X. Qiu, “Performance review on llm for solving leetcode problems,” in *2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM)*. IEEE, 2024, pp. 1050–1054.
- [80] A. Z. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, “Vert: Verified equivalent rust transpilation with large language models as few-shot learners,” *arXiv preprint arXiv:2404.18852*, 2024.
- [81] J. Yang and J. Leskovec, “Defining and evaluating network communities based on ground-truth,” in *Proceedings of the ACM SIGKDD workshop on mining data semantics*, 2012, pp. 1–8.
- [82] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, “Reevo: Large language models as hyper-heuristics with reflective evolution,” *Advances in neural information processing systems*, vol. 37, pp. 43 571–43 608, 2024.
- [83] T. Ye, T. Ma, X. Zhang, H. Yu, J. Yin, and W. Wang, “A problem-oriented perspective and anchor verification for code optimization,” 2026. [Online]. Available: <https://arxiv.org/abs/2406.11935>
- [84] G. Zhang, S. Zhu, A. Wei, Z. Song, A. Nie, Z. Jia, N. Vijaykumar, Y. Wang, and K. Olukotun, “Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization,” *arXiv preprint arXiv:2511.15915*, 2025.
- [85] H. Zhang, C. David, M. Wang, B. Paulsen, and D. Kroening, “Scalable, validated code translation of entire projects using large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.08035>

- [86] H. Zhang, C. David, Y. Yu, and M. Wang, “Ownership guided c to rust translation,” in *International Conference on Computer Aided Verification*. Springer, pp. 459–482.
- [87] Q. Zhang, J. Wang, G. H. Xu, and M. Kim, “Heterogen: transpiling c to heterogeneous hls code with automated test generation and program repair,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 1017–1029.
- [88] W. Zhang, H. Chen, C. Jiang, and L. Zhu, “Improvement and experimental evaluation bellman-ford algorithm,” in *2013 International Conference on Advanced ICT and Education (ICAICTE-13)*. Atlantis Press, 2013, pp. 138–141.
- [89] W. Zhang, J. Wang, L. Liang, Y. Zhao, H. Wen, and Z. Zhao, “Evocodebench: A human-performance benchmark for self-evolving llm-driven coding systems,” *arXiv preprint arXiv:2602.10171*, 2026.
- [90] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu, “Towards understanding the runtime performance of rust,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [91] Q. Zhou, Y. Wen, R. Chen, K. Gao, W. Xiong, L. Li, Q. Guo, Y. Wu, and Y. Chen, “Qimeng-gemm: Automatically generating high-performance matrix multiplication code by exploiting large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 21, 2025, pp. 22 982–22 990.
- [92] T. Zhou, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, “Llm-driven multi-step translation from c to rust using static analysis,” *arXiv preprint arXiv:2503.12511*, 2025.